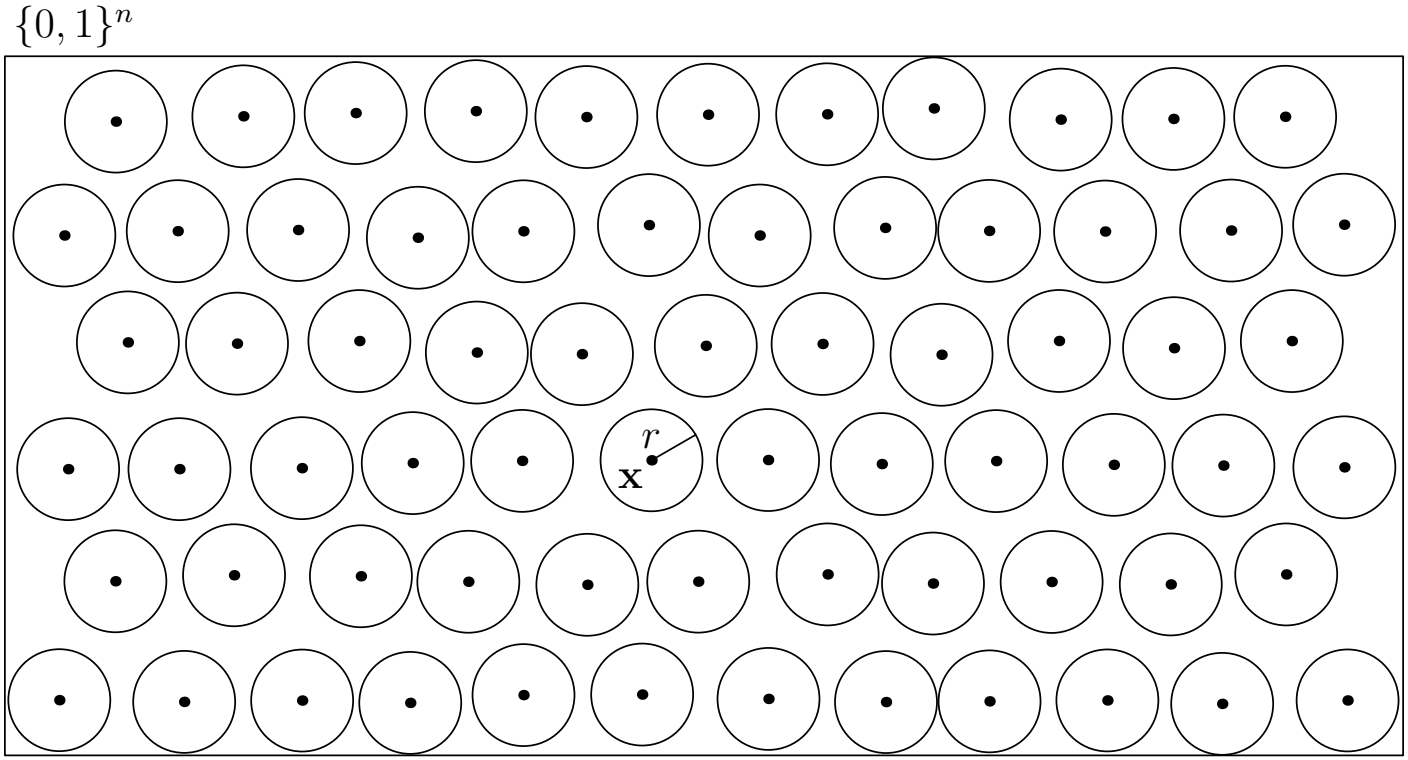
Mathematics of Machine Learning

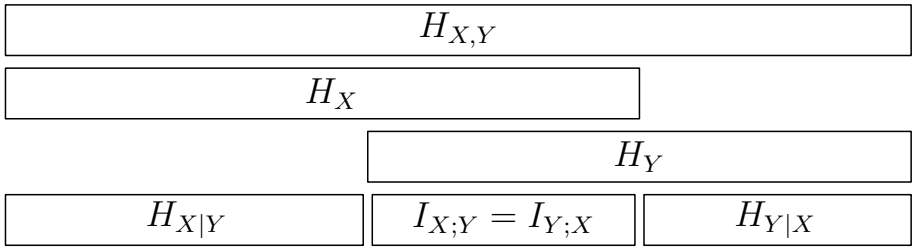
Information Theory



Information, KL-divergence, Minimum Description Length

Outline

1. **Information Theory**
2. KL-Divergence
3. Minimum Description Length
4. Variational Auto-Encoders



Communicating Via a Noisy Channel

- Information theory considers communicating down a (noisy) channel

$$X \sim \mathbb{P}(X) \xrightarrow{\text{noisy channel}} Y \sim \mathbb{P}(Y \mid X)$$

- We send a message X (with probability $\mathbb{P}(X)$) and receive a message Y with probability $\mathbb{P}(Y \mid X)$

- The uncertainty of the message sent, given we received a message y is

$$H_{X|Y=y} = -\sum_{x \in \mathcal{X}} \mathbb{P}(X = x \mid Y = y) \log(\mathbb{P}(X = x \mid Y = y))$$

- The expected uncertainty in the message sent is

$$H_{X|Y} = \sum_{y \in \mathcal{Y}} \mathbb{P}(Y = y) H_{X|Y=y} = -\sum_{x,y} \mathbb{P}(X = x, Y = y) \log(\mathbb{P}(X = x \mid Y = y))$$

Communicating Via a Noisy Channel

- Information theory considers communicating down a (noisy) channel

$$X \sim \mathbb{P}(X) \xrightarrow{\text{noisy channel}} Y \sim \mathbb{P}(Y \mid X)$$

- We send a message X (with probability $\mathbb{P}(X)$) and receive a message Y with probability $\mathbb{P}(Y \mid X)$

- The uncertainty of the message sent, given we received a message y is

$$H_{X|Y=y} = -\sum_{x \in \mathcal{X}} \mathbb{P}(X = x \mid Y = y) \log(\mathbb{P}(X = x \mid Y = y))$$

- The expected uncertainty in the message sent is

$$H_{X|Y} = \sum_{y \in \mathcal{Y}} \mathbb{P}(Y = y) H_{X|Y=y} = -\sum_{x,y} \mathbb{P}(X = x, Y = y) \log(\mathbb{P}(X = x \mid Y = y))$$

Communicating Via a Noisy Channel

- Information theory considers communicating down a (noisy) channel

$$X \sim \mathbb{P}(X) \xrightarrow{\text{noisy channel}} Y \sim \mathbb{P}(Y \mid X)$$

- We send a message X (with probability $\mathbb{P}(X)$) and receive a message Y with probability $\mathbb{P}(Y \mid X)$

- The uncertainty of the message sent, given we received a message y is

$$H_{X|Y=y} = -\sum_{x \in \mathcal{X}} \mathbb{P}(X = x \mid Y = y) \log(\mathbb{P}(X = x \mid Y = y))$$

- The expected uncertainty in the message sent is

$$H_{X|Y} = \sum_{y \in \mathcal{Y}} \mathbb{P}(Y = y) H_{X|Y=y} = -\sum_{x,y} \mathbb{P}(X = x, Y = y) \log(\mathbb{P}(X = x \mid Y = y))$$

Joint Entropy

- We can define the **joint entropy**

$$H_{X,Y} = -\sum_{x,y} P_{X,Y}(x,y)\log(P_{X,Y}(x,y))$$

- If the message we receive is independent of the message that is sent then $H_{X,Y} = H_X + H_Y$ (we saw this in the last lecture)
- $H_{X,Y} < H_X + H_Y$ if X and Y are dependent
- Since $\mathbb{P}(X,Y) = \mathbb{P}(Y|X)\mathbb{P}(X) = \mathbb{P}(X|Y)\mathbb{P}(Y)$ it follows

$$H_{X,Y} = H_X + H_{Y|X} = H_Y + H_{X|Y}$$

- Or $H_X - H_{X|Y} = H_Y - H_{Y|X}$

Joint Entropy

- We can define the **joint entropy**

$$H_{X,Y} = -\sum_{x,y} P_{X,Y}(x,y)\log(P_{X,Y}(x,y))$$

- If the message we receive is independent of the message that is sent then $H_{X,Y} = H_X + H_Y$ (we saw this in the last lecture)
- $H_{X,Y} < H_X + H_Y$ if X and Y are dependent
- Since $\mathbb{P}(X,Y) = \mathbb{P}(Y|X)\mathbb{P}(X) = \mathbb{P}(X|Y)\mathbb{P}(Y)$ it follows

$$H_{X,Y} = H_X + H_{Y|X} = H_Y + H_{X|Y}$$

- Or $H_X - H_{X|Y} = H_Y - H_{Y|X}$

Joint Entropy

- We can define the **joint entropy**

$$H_{X,Y} = -\sum_{x,y} P_{X,Y}(x,y)\log(P_{X,Y}(x,y))$$

- If the message we receive is independent of the message that is sent then $H_{X,Y} = H_X + H_Y$ (we saw this in the last lecture)
- $H_{X,Y} < H_X + H_Y$ if X and Y are dependent
- Since $\mathbb{P}(X,Y) = \mathbb{P}(Y|X)\mathbb{P}(X) = \mathbb{P}(X|Y)\mathbb{P}(Y)$ it follows

$$H_{X,Y} = H_X + H_{Y|X} = H_Y + H_{X|Y}$$

- Or $H_X - H_{X|Y} = H_Y - H_{Y|X}$

Joint Entropy

- We can define the **joint entropy**

$$H_{X,Y} = -\sum_{x,y} P_{X,Y}(x,y)\log(P_{X,Y}(x,y))$$

- If the message we receive is independent of the message that is sent then $H_{X,Y} = H_X + H_Y$ (we saw this in the last lecture)
- $H_{X,Y} < H_X + H_Y$ if X and Y are dependent
- Since $\mathbb{P}(X,Y) = \mathbb{P}(Y|X)\mathbb{P}(X) = \mathbb{P}(X|Y)\mathbb{P}(Y)$ it follows

$$H_{X,Y} = H_X + H_{Y|X} = H_Y + H_{X|Y}$$

- Or $H_X - H_{X|Y} = H_Y - H_{Y|X}$

Joint Entropy

- We can define the **joint entropy**

$$H_{X,Y} = -\sum_{x,y} P_{X,Y}(x,y)\log(P_{X,Y}(x,y))$$

- If the message we receive is independent of the message that is sent then $H_{X,Y} = H_X + H_Y$ (we saw this in the last lecture)
- $H_{X,Y} < H_X + H_Y$ if X and Y are dependent
- Since $\mathbb{P}(X,Y) = \mathbb{P}(Y|X)\mathbb{P}(X) = \mathbb{P}(X|Y)\mathbb{P}(Y)$ it follows

$$H_{X,Y} = H_X + H_{Y|X} = H_Y + H_{X|Y}$$

- Or $H_X - H_{X|Y} = H_Y - H_{Y|X}$

Mutual Information

- The amount of uncertainty about the message being sent, X, before receiving the message is $H_X = -\mathbb{E}_X[\log \mathbb{P}(X)]$

- Shannon defined the *mutual information* to be the expected loss in uncertainty when we receive a message

$$I_{X;Y} = H_X - H_{X|Y}$$

- Since $H_X - H_{X|Y} = H_Y - H_{Y|X}$ it follows

$$I_{X;Y} = I_{Y;X}$$

Mutual Information

- The amount of uncertainty about the message being sent, X, before receiving the message is $H_X = -\mathbb{E}_X[\log \mathbb{P}(X)]$
- Shannon defined the *mutual information* to be the expected loss in uncertainty when we receive a message

$$I_{X;Y} = H_X - H_{X|Y}$$

- Since $H_X - H_{X|Y} = H_Y - H_{Y|X}$ it follows

$$I_{X;Y} = I_{Y;X}$$

Mutual Information

- The amount of uncertainty about the message being sent, X, before receiving the message is $H_X = -\mathbb{E}_X[\log \mathbb{P}(X)]$
- Shannon defined the *mutual information* to be the expected loss in uncertainty when we receive a message

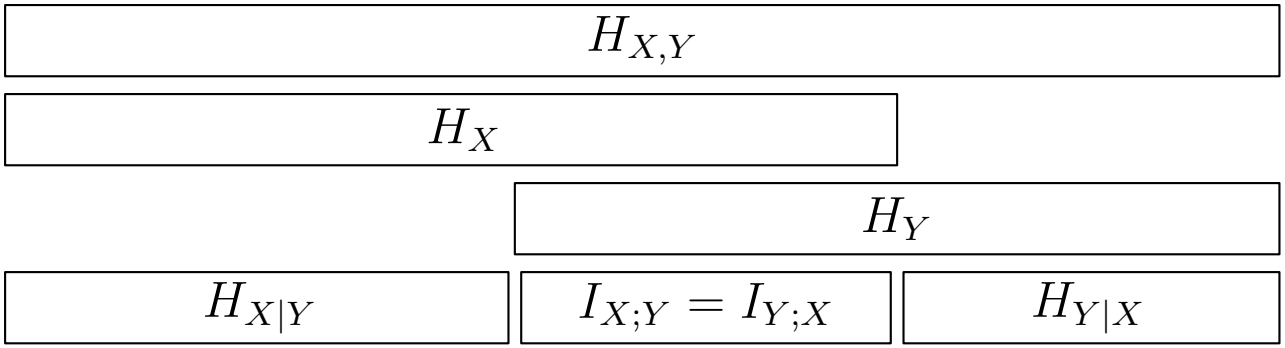
$$I_{X;Y} = H_X - H_{X|Y}$$

- Since $H_X - H_{X|Y} = H_Y - H_{Y|X}$ it follows

$$I_{X;Y} = I_{Y;X}$$

Channel Capacity

- We can summarise these relationships diagrammatically



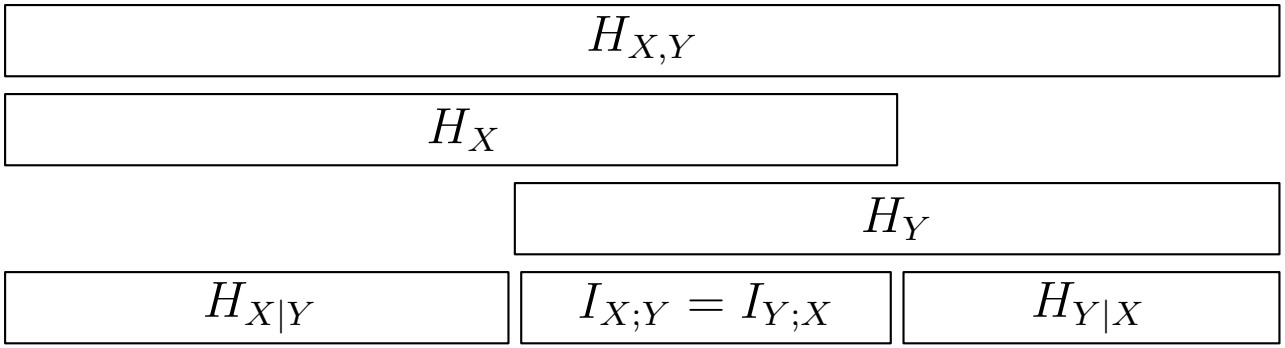
- Shannon defined the *capacity* of a noisy channel as

$$C = \max_{\mathbb{P}(X)} I_{X;Y}$$

- That is, you choose the probability distribution of the message to maximise the information gain

Channel Capacity

- We can summarise these relationships diagrammatically



- Shannon defined the *capacity* of a noisy channel as

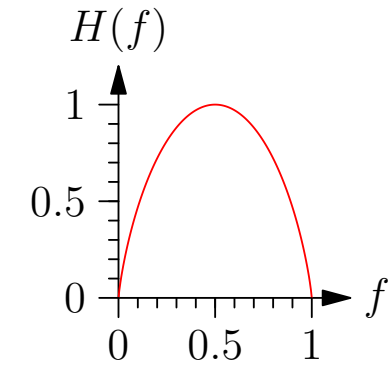
$$C = \max_{\mathbb{P}(X)} I_{X;Y}$$

- That is, you choose the probability distribution of the message to maximise the information gain

Independent Noise

- The simplest model of a noisy channel is a binary channel where each symbol is corrupted independently with a probability f

$$\mathbb{P}(Y = 1|X = 0) = \mathbb{P}(Y = 0|X = 1) = f$$



- An elementary calculation shows that

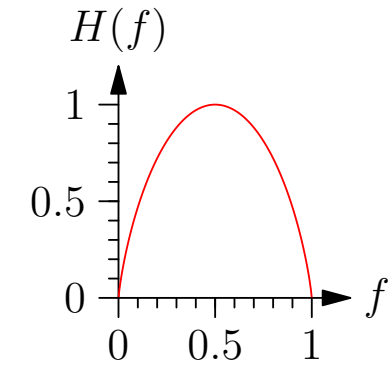
$$H_{Y_i|X_i} = -(1-f)\log_2(1-f) - f\log_2(f) = H(f)$$

- So $I_{X;Y} = H_Y - H_{Y|X} \leq 1 - H(f)$ per bit, with equality when 0 and 1 are sent equally often

Independent Noise

- The simplest model of a noisy channel is a binary channel where each symbol is corrupted independently with a probability f

$$\mathbb{P}(Y = 1|X = 0) = \mathbb{P}(Y = 0|X = 1) = f$$



- An elementary calculation shows that

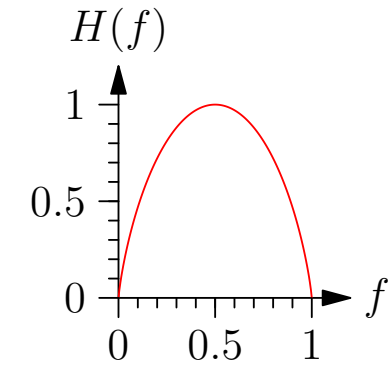
$$H_{Y_i|X_i} = -(1-f)\log_2(1-f) - f\log_2(f) = H(f)$$

- So $I_{X;Y} = H_Y - H_{Y|X} \leq 1 - H(f)$ per bit, with equality when 0 and 1 are sent equally often

Independent Noise

- The simplest model of a noisy channel is a binary channel where each symbol is corrupted independently with a probability f

$$\mathbb{P}(Y = 1|X = 0) = \mathbb{P}(Y = 0|X = 1) = f$$



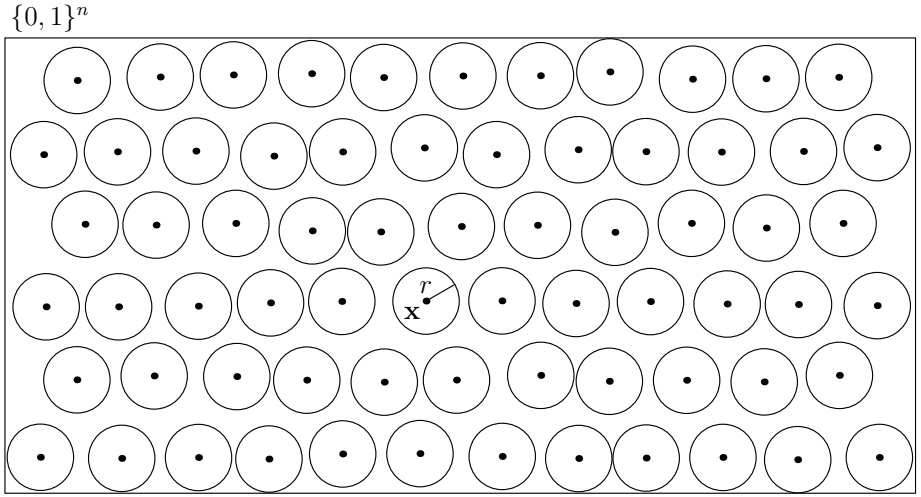
- An elementary calculation shows that

$$H_{Y_i|X_i} = -(1-f)\log_2(1-f) - f\log_2(f) = H(f)$$

- So $I_{X;Y} = H_Y - H_{Y|X} \leq 1 - H(f)$ per bit, with equality when 0 and 1 are sent equally often

Error Correcting Codes

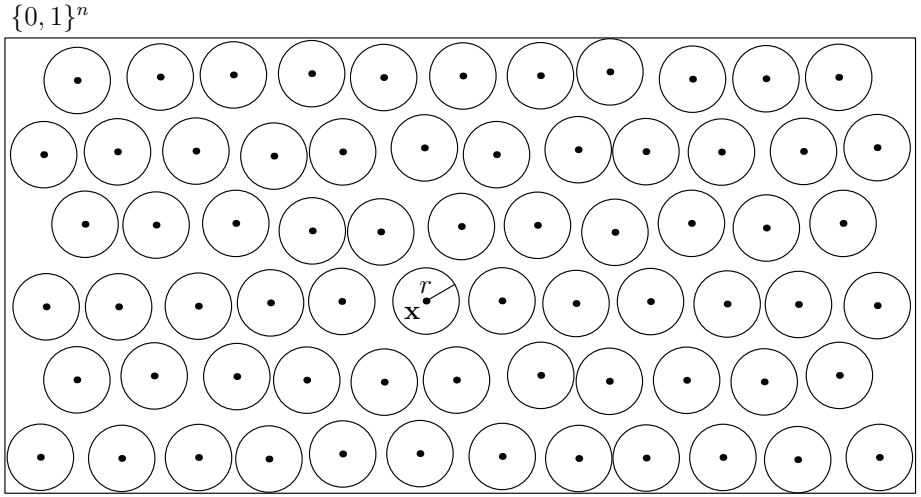
- To reduce the chance of misinterpreting a message we need to build an error correcting code
- We can do this dividing the space of binary messages into a set of Hamming balls



- A Hamming ball $B(\boldsymbol{x},r)$ is the set of strings that differ from n-dimensional binary string, $\boldsymbol{x}$, by at most r digits

Error Correcting Codes

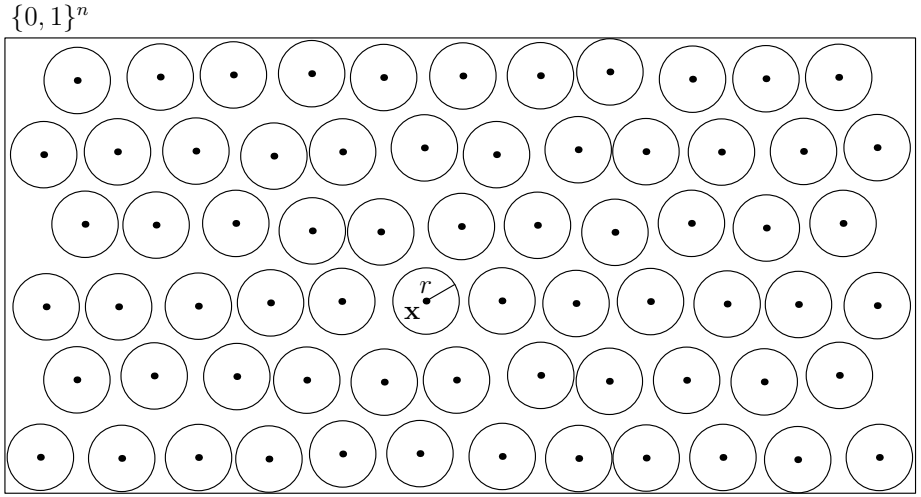
- To reduce the chance of misinterpreting a message we need to build an error correcting code
- We can do this dividing the space of binary messages into a set of Hamming balls



- A Hamming ball $B(\boldsymbol{x},r)$ is the set of strings that differ from n-dimensional binary string, $\boldsymbol{x}$, by at most r digits

Error Correcting Codes

- To reduce the chance of misinterpreting a message we need to build an error correcting code
- We can do this dividing the space of binary messages into a set of Hamming balls



- A Hamming ball $B(\boldsymbol{x},r)$ is the set of strings that differ from n-dimensional binary string, $\boldsymbol{x}$, by at most r digits

Volume of Coding Space

- The expected number of errors in a string of length n given an error rate of f is nf
- For sufficiently large n the number of errors is, with high probability, less than $(f+\epsilon)n$ (for $\epsilon > 0$)
- If we make the radius of the Hamming ball $r = (f+\epsilon)n$ $(\epsilon > 0)$ then we would expect no error for sufficiently large n
- An upper bound on the number of code words we can send in a string of length n is

$$\frac{2^n}{|B(\boldsymbol{x}_i, r)|} = \frac{2^n}{\binom{n}{r}}$$

Volume of Coding Space

- The expected number of errors in a string of length n given an error rate of f is nf
- For sufficiently large n the number of errors is, with high probability, less than $(f+\epsilon)n$ (for $\epsilon > 0$)
- If we make the radius of the Hamming ball $r = (f+\epsilon)n$ $(\epsilon > 0)$ then we would expect no error for sufficiently large n
- An upper bound on the number of code words we can send in a string of length n is

$$\frac{2^n}{|B(\boldsymbol{x}_i, r)|} = \frac{2^n}{\binom{n}{r}}$$

Volume of Coding Space

- The expected number of errors in a string of length n given an error rate of f is nf
- For sufficiently large n the number of errors is, with high probability, less than $(f+\epsilon)n$ (for $\epsilon > 0$)
- If we make the radius of the Hamming ball $r = (f+\epsilon)n$ $(\epsilon > 0)$ then we would expect no error for sufficiently large n
- An upper bound on the number of code words we can send in a string of length n is

$$\frac{2^n}{|B(\boldsymbol{x}_i, r)|} = \frac{2^n}{\binom{n}{r}}$$

Volume of Coding Space

- The expected number of errors in a string of length n given an error rate of f is nf
- For sufficiently large n the number of errors is, with high probability, less than $(f+\epsilon)n$ (for $\epsilon > 0$)
- If we make the radius of the Hamming ball $r = (f+\epsilon)n$ $(\epsilon > 0)$ then we would expect no error for sufficiently large n
- An upper bound on the number of code words we can send in a string of length n is

$$\frac{2^n}{|B(\boldsymbol{x}_i, r)|} = \frac{2^n}{\binom{n}{r}}$$

Volume of Coding Space

- The expected number of errors in a string of length n given an error rate of f is nf
- For sufficiently large n the number of errors is, with high probability, less than $(f+\epsilon)n$ (for $\epsilon > 0$)
- If we make the radius of the Hamming ball $r = (f+\epsilon)n$ ($\epsilon > 0$) then we would expect no error for sufficiently large n
- An upper bound on the number of code words we can send in a string of length n is

$$\frac{2^n}{|B(\boldsymbol{x}_i, r)|} = \frac{2^n}{\binom{n}{r}} = 2^n \mathrm{e}^{-\log(n!)+\log(r!)+\log((n-r)!)}$$

Volume of Coding Space

- The expected number of errors in a string of length n given an error rate of f is nf
- For sufficiently large n the number of errors is, with high probability, less than $(f+\epsilon)n$ (for $\epsilon > 0$)
- If we make the radius of the Hamming ball $r = (f+\epsilon)n$ ($\epsilon > 0$) then we would expect no error for sufficiently large n
- An upper bound on the number of code words we can send in a string of length n is

$$\frac{2^n}{|B(\boldsymbol{x}_i, r)|} = \frac{2^n}{\binom{n}{r}} = 2^n \mathrm{e}^{-\log(n!)+\log(r!)+\log((n-r)!)} \approx c\sqrt{n}2^{n(1-H(f+\epsilon))}$$

Shannon's Bound

- So we can send at most about $2^{n(1-H(f))}$ distinguishable messages: a rate of $1 - H(f)$ bits per bit sent
- Shannon also showed that about this many code words chosen at random can be decoded with almost no errors
- The maximum rate is the channel capacity, here $C = 1 - H(f)$
- If $f = 0.1$ then $C = 1 - 0.469 = 0.531\text{bits}$, so we need codes about 1.9 times as long to communicate accurately over a noisy channel with a 10% corruption rate
- Random codes cannot be decoded efficiently, but modern codes (turbo, LDPC and polar codes, used in 4G, 5G and Wi-Fi) come close to Shannon's bound

Shannon's Bound

- So we can send at most about $2^{n(1-H(f))}$ distinguishable messages: a rate of $1 - H(f)$ bits per bit sent
- Shannon also showed that about this many code words chosen at random can be decoded with almost no errors
- The maximum rate is the channel capacity, here $C = 1 - H(f)$
- If $f = 0.1$ then $C = 1 - 0.469 = 0.531\text{bits}$, so we need codes about 1.9 times as long to communicate accurately over a noisy channel with a 10% corruption rate
- Random codes cannot be decoded efficiently, but modern codes (turbo, LDPC and polar codes, used in 4G, 5G and Wi-Fi) come close to Shannon's bound

Shannon's Bound

- So we can send at most about $2^{n(1-H(f))}$ distinguishable messages: a rate of $1 - H(f)$ bits per bit sent
- Shannon also showed that about this many code words chosen at random can be decoded with almost no errors
- The maximum rate is the channel capacity, here $C = 1 - H(f)$
- If $f = 0.1$ then $C = 1 - 0.469 = 0.531\text{bits}$, so we need codes about 1.9 times as long to communicate accurately over a noisy channel with a 10% corruption rate
- Random codes cannot be decoded efficiently, but modern codes (turbo, LDPC and polar codes, used in 4G, 5G and Wi-Fi) come close to Shannon's bound

Shannon's Bound

- So we can send at most about $2^{n(1-H(f))}$ distinguishable messages: a rate of $1 - H(f)$ bits per bit sent
- Shannon also showed that about this many code words chosen at random can be decoded with almost no errors
- The maximum rate is the channel capacity, here $C = 1 - H(f)$
- If $f = 0.1$ then $C = 1 - 0.469 = 0.531\text{bits}$, so we need codes about 1.9 times as long to communicate accurately over a noisy channel with a 10% corruption rate
- Random codes cannot be decoded efficiently, but modern codes (turbo, LDPC and polar codes, used in 4G, 5G and Wi-Fi) come close to Shannon's bound

Shannon's Bound

- So we can send at most about $2^{n(1-H(f))}$ distinguishable messages: a rate of $1 - H(f)$ bits per bit sent
- Shannon also showed that about this many code words chosen at random can be decoded with almost no errors
- The maximum rate is the channel capacity, here $C = 1 - H(f)$
- If $f = 0.1$ then $C = 1 - 0.469 = 0.531\,\text{bits}$, so we need codes about 1.9 times as long to communicate accurately over a noisy channel with a 10% corruption rate
- Random codes cannot be decoded efficiently, but modern codes (turbo, LDPC and polar codes, used in 4G, 5G and Wi-Fi) come close to Shannon's bound

Using Mutual Information

- Mutual information is used quite often in machine learning
- The **information gain** used to choose splits in decision trees is the mutual information between a feature and the label
- Suppose we want to align two images (e.g. an MRI and a CT scan of the same patient) through some non-linear transformations
- One way of doing this is to choose the non-linear transformations that maximise the mutual information (or normalised mutual information) between the two images

Using Mutual Information

- Mutual information is used quite often in machine learning
- The **information gain** used to choose splits in decision trees is the mutual information between a feature and the label
- Suppose we want to align two images (e.g. an MRI and a CT scan of the same patient) through some non-linear transformations
- One way of doing this is to choose the non-linear transformations that maximise the mutual information (or normalised mutual information) between the two images

Using Mutual Information

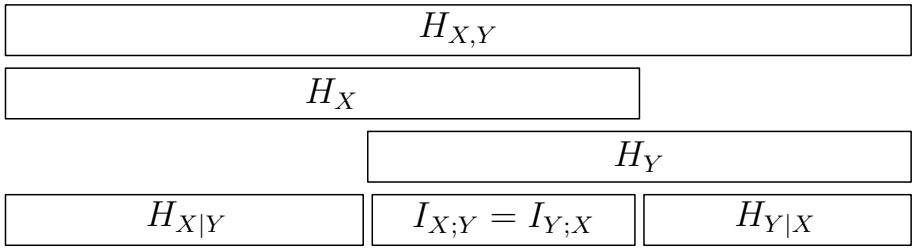
- Mutual information is used quite often in machine learning
- The **information gain** used to choose splits in decision trees is the mutual information between a feature and the label
- Suppose we want to align two images (e.g. an MRI and a CT scan of the same patient) through some non-linear transformations
- One way of doing this is to choose the non-linear transformations that maximise the mutual information (or normalised mutual information) between the two images

Using Mutual Information

- Mutual information is used quite often in machine learning
- The **information gain** used to choose splits in decision trees is the mutual information between a feature and the label
- Suppose we want to align two images (e.g. an MRI and a CT scan of the same patient) through some non-linear transformations
- One way of doing this is to choose the non-linear transformations that maximise the mutual information (or normalised mutual information) between the two images

Outline

1. Information Theory
2. **KL-Divergence**
3. Minimum Description Length
4. Variational Auto-Encoders



KL-Divergence

- The Kullback-Leibler (KL) divergence is

$$\begin{aligned}\mathrm{KL}\big(p\big\|q\big) &= \mathbb{E}_{X\sim p(X)}\left[\log\left(\frac{p(X)}{q(X)}\right)\right] \\ &= \mathbb{E}_{X\sim p(X)}[-\log(q(X))] - H_X\end{aligned}$$

- Recall $-\log(q(X=x))$ is the length of code needed to send a message x with a probability $q(X=x)$
- Thus $-\mathbb{E}_{X\sim p(X)}[\log(q(X))]$ is the expected length of message needed to code $X\sim p(X)$ using the optimal code for the distribution $q(X)$ rather than $p(X)$
- $\mathrm{KL}\big(p\big\|q\big)$ is also known as the **relative entropy** and measures the expected extra length in coding $X\sim p(X)$ if we use the wrong distribution $q(X)$

KL-Divergence

- The Kullback-Leibler (KL) divergence is

$$\begin{aligned}\mathrm{KL}\big(p\big\|q\big) &= \mathbb{E}_{X\sim p(X)}\left[\log\left(\frac{p(X)}{q(X)}\right)\right] \\ &= \mathbb{E}_{X\sim p(X)}[-\log(q(X))] - H_X\end{aligned}$$

- Recall $-\log(q(X=x))$ is the length of code needed to send a message x with a probability $q(X=x)$

- Thus $-\mathbb{E}_{X\sim p(X)}[\log(q(X))]$ is the expected length of message needed to code $X\sim p(X)$ using the optimal code for the distribution $q(X)$ rather than $p(X)$

- $\mathrm{KL}\big(p\big\|q\big)$ is also known as the **relative entropy** and measures the expected extra length in coding $X\sim p(X)$ if we use the wrong distribution $q(X)$

KL-Divergence

- The Kullback-Leibler (KL) divergence is

$$\begin{aligned}\mathrm{KL}\big(p\big\|q\big) &= \mathbb{E}_{X\sim p(X)}\left[\log\left(\frac{p(X)}{q(X)}\right)\right] \\ &= \mathbb{E}_{X\sim p(X)}[-\log(q(X))] - H_X\end{aligned}$$

- Recall $-\log(q(X=x))$ is the length of code needed to send a message x with a probability $q(X=x)$

- Thus $-\mathbb{E}_{X\sim p(X)}[\log(q(X))]$ is the expected length of message needed to code $X\sim p(X)$ using the optimal code for the distribution $q(X)$ rather than $p(X)$

- $\mathrm{KL}\big(p\big\|q\big)$ is also known as the **relative entropy** and measures the expected extra length in coding $X\sim p(X)$ if we use the wrong distribution $q(X)$

KL-Divergence

- The Kullback-Leibler (KL) divergence is

$$\begin{aligned}\mathrm{KL}\big(p\big\|q\big) &= \mathbb{E}_{X\sim p(X)}\left[\log\left(\frac{p(X)}{q(X)}\right)\right] \\ &= \mathbb{E}_{X\sim p(X)}[-\log(q(X))] - H_X\end{aligned}$$

- Recall $-\log(q(X=x))$ is the length of code needed to send a message x with a probability $q(X=x)$

- Thus $-\mathbb{E}_{X\sim p(X)}[\log(q(X))]$ is the expected length of message needed to code $X\sim p(X)$ using the optimal code for the distribution $q(X)$ rather than $p(X)$

- $\mathrm{KL}\big(p\big\|q\big)$ is also known as the **relative entropy** and measures the expected extra length in coding $X\sim p(X)$ if we use the wrong distribution $q(X)$

Cross-Entropy Loss

- A classifier gives probabilities $q(y|\boldsymbol{x})$; it is usually trained by minimising the **cross-entropy**

$$\mathcal{L} = -\frac{1}{m}\sum_{k=1}^{m}\log(q(y_k|\boldsymbol{x}_k))$$

- It is the average code length for the labels using the model's probabilities
- For labels from the true $p(y|\boldsymbol{x})$, at each $\boldsymbol{x}$

$$\mathbb{E}_{Y\sim p}[-\log(q(Y|\boldsymbol{x}))] = H_p + \mathrm{KL}\big(p\big\|q\big)$$

- H_p does not depend on the model: minimising cross-entropy minimises $\mathrm{KL}\big(p\big\|q\big)$

Cross-Entropy Loss

- A classifier gives probabilities $q(y|\boldsymbol{x})$; it is usually trained by minimising the **cross-entropy**

$$\mathcal{L} = -\frac{1}{m}\sum_{k=1}^{m}\log(q(y_k|\boldsymbol{x}_k))$$

- It is the average code length for the labels using the model's probabilities
- For labels from the true $p(y|\boldsymbol{x})$, at each $\boldsymbol{x}$

$$\mathbb{E}_{Y\sim p}[-\log(q(Y|\boldsymbol{x}))] = H_p + \mathrm{KL}\big(p\big\|q\big)$$

- H_p does not depend on the model: minimising cross-entropy minimises $\mathrm{KL}\big(p\big\|q\big)$

Cross-Entropy Loss

- A classifier gives probabilities $q(y|\boldsymbol{x})$; it is usually trained by minimising the **cross-entropy**

$$\mathcal{L} = -\frac{1}{m}\sum_{k=1}^{m}\log(q(y_k|\boldsymbol{x}_k))$$

- It is the average code length for the labels using the model's probabilities
- For labels from the true $p(y|\boldsymbol{x})$, at each $\boldsymbol{x}$

$$\mathbb{E}_{Y\sim p}[-\log(q(Y|\boldsymbol{x}))] = H_p + \mathrm{KL}\big(p\big\|q\big)$$

- H_p does not depend on the model: minimising cross-entropy minimises $\mathrm{KL}\big(p\big\|q\big)$

Cross-Entropy Loss

- A classifier gives probabilities $q(y|\boldsymbol{x})$; it is usually trained by minimising the **cross-entropy**

$$\mathcal{L} = -\frac{1}{m}\sum_{k=1}^{m}\log(q(y_k|\boldsymbol{x}_k))$$

- It is the average code length for the labels using the model's probabilities
- For labels from the true $p(y|\boldsymbol{x})$, at each $\boldsymbol{x}$

$$\mathbb{E}_{Y\sim p}[-\log(q(Y|\boldsymbol{x}))] = H_p + \mathrm{KL}\big(p\big\|q\big)$$

- H_p does not depend on the model: minimising cross-entropy minimises $\mathrm{KL}\big(p\big\|q\big)$

Variational Approximation

- Recall we use MCMC in Bayesian inference because the posterior distribution is too complicated to write down in closed form
- In the variational approximation we approximate the posterior distribution by a simpler (typically factored distribution), e.g.

$$f(\boldsymbol{\theta} \mid \mathcal{D}) \approx g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) = \prod_i g(\theta_i \mid \phi_i)$$

- The standard method is to maximise the **evidence lower bound** (ELBO, minus the **variational free energy**)

$$\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta}, \mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta}$$

Variational Approximation

- Recall we use MCMC in Bayesian inference because the posterior distribution is too complicated to write down in closed form
- In the variational approximation we approximate the posterior distribution by a simpler (typically factored distribution), e.g.

$$f(\boldsymbol{\theta} \mid \mathcal{D}) \approx g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) = \prod_i g(\theta_i \mid \phi_i)$$

- The standard method is to maximise the **evidence lower bound** (ELBO, minus the **variational free energy**)

$$\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta}, \mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta}$$

Variational Approximation

- Recall we use MCMC in Bayesian inference because the posterior distribution is too complicated to write down in closed form
- In the variational approximation we approximate the posterior distribution by a simpler (typically factored distribution), e.g.

$$f(\boldsymbol{\theta} \mid \mathcal{D}) \approx g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) = \prod_i g(\theta_i \mid \phi_i)$$

- The standard method is to maximise the **evidence lower bound** (ELBO, minus the **variational free energy**)

$$\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta}, \mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta}$$

Evidence Lower Bound (ELBO)

- We can re-write the ELBO as

$$\begin{aligned}\Phi(\boldsymbol{\phi}) &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{(f(\boldsymbol{\theta},\mathcal{D})/f(\mathcal{D}))\, f(\mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta} \\ &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \left(\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta} \mid \mathcal{D})}\right) - \log(f(\mathcal{D}))\right) \mathrm{d}\boldsymbol{\theta} \\ &= -\mathrm{KL}\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big\| f(\boldsymbol{\theta} \mid \mathcal{D})\big) + \log(f(\mathcal{D}))\end{aligned}$$

- If we maximise $\Phi(\boldsymbol{\phi})$, we end up minimising the KL divergence between g and f so that $g \approx f$ and $\Phi(\boldsymbol{\phi}) \approx \log(f(\mathcal{D}))$

- That is, we choose the parameters of our simple factorised distribution so that it is close to the true posterior

Evidence Lower Bound (ELBO)

- We can re-write the ELBO as

$$\begin{aligned}\Phi(\boldsymbol{\phi}) &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{(f(\boldsymbol{\theta}, \mathcal{D})/f(\mathcal{D}))f(\mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta} \\ &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \left(\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta} \mid \mathcal{D})}\right) - \log(f(\mathcal{D}))\right) \mathrm{d}\boldsymbol{\theta} \\ &= -\mathrm{KL}\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big\| f(\boldsymbol{\theta} \mid \mathcal{D})\big) + \log(f(\mathcal{D}))\end{aligned}$$

- If we maximise $\Phi(\boldsymbol{\phi})$, we end up minimising the KL divergence between g and f so that $g \approx f$ and $\Phi(\boldsymbol{\phi}) \approx \log(f(\mathcal{D}))$

- That is, we choose the parameters of our simple factorised distribution so that it is close to the true posterior

Evidence Lower Bound (ELBO)

- We can re-write the ELBO as

$$\begin{aligned}\Phi(\boldsymbol{\phi}) &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{(f(\boldsymbol{\theta},\mathcal{D})/f(\mathcal{D}))f(\mathcal{D})}\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\left(\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta} \mid \mathcal{D})}\right) - \log(f(\mathcal{D}))\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\mathrm{KL}\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big\| f(\boldsymbol{\theta} \mid \mathcal{D})\big) + \log(f(\mathcal{D}))\end{aligned}$$

- If we maximise $\Phi(\boldsymbol{\phi})$, we end up minimising the KL divergence between g and f so that $g \approx f$ and $\Phi(\boldsymbol{\phi}) \approx \log(f(\mathcal{D}))$

- That is, we choose the parameters of our simple factorised distribution so that it is close to the true posterior

Evidence Lower Bound (ELBO)

- We can re-write the ELBO as

$$\begin{aligned}\Phi(\boldsymbol{\phi}) &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{(f(\boldsymbol{\theta},\mathcal{D})/f(\mathcal{D}))f(\mathcal{D})}\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\left(\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta} \mid \mathcal{D})}\right) - \log(f(\mathcal{D}))\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\mathrm{KL}\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big\| f(\boldsymbol{\theta} \mid \mathcal{D})\big) + \log(f(\mathcal{D}))\end{aligned}$$

- If we maximise $\Phi(\boldsymbol{\phi})$, we end up minimising the KL divergence between g and f so that $g \approx f$ and $\Phi(\boldsymbol{\phi}) \approx \log(f(\mathcal{D}))$

- That is, we choose the parameters of our simple factorised distribution so that it is close to the true posterior

Evidence Lower Bound (ELBO)

- We can re-write the ELBO as

$$\begin{aligned}\Phi(\boldsymbol{\phi}) &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{(f(\boldsymbol{\theta},\mathcal{D})/f(\mathcal{D}))f(\mathcal{D})}\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\left(\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta} \mid \mathcal{D})}\right) - \log(f(\mathcal{D}))\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\mathrm{KL}\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big\| f(\boldsymbol{\theta} \mid \mathcal{D})\big) + \log(f(\mathcal{D}))\end{aligned}$$

- If we maximise $\Phi(\boldsymbol{\phi})$, we end up minimising the KL divergence between g and f so that $g \approx f$ and $\Phi(\boldsymbol{\phi}) \approx \log(f(\mathcal{D}))$

- That is, we choose the parameters of our simple factorised distribution so that it is close to the true posterior

Put Another Way

- We can rewrite the ELBO as
  $\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta}|\boldsymbol{\phi})}{f(\boldsymbol{\theta},\mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta} = L_g(\boldsymbol{\phi}) + H_g(\boldsymbol{\phi})$ where

$$L_g(\boldsymbol{\phi}) = \int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big(\log\big(f(\mathcal{D}|\boldsymbol{\theta}) f(\boldsymbol{\theta})\big)\big) \mathrm{d}\boldsymbol{\theta}$$

  is the expected log of likelihood $\times$ prior, which is large when the data is well modelled (we put $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$ where $f(\boldsymbol{\theta},\mathcal{D})$ is large, c.f. MAP)

- The second term is an entropy

$$H_g(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\big) \mathrm{d}\boldsymbol{\theta}$$

  That is, we maximise the uncertainty of the distribution $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$

Put Another Way

- We can rewrite the ELBO as
  $\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta}|\boldsymbol{\phi})}{f(\boldsymbol{\theta},\mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta} = L_g(\boldsymbol{\phi}) + H_g(\boldsymbol{\phi})$ where

$$L_g(\boldsymbol{\phi}) = \int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big(\log\big(f(\mathcal{D}|\boldsymbol{\theta}) f(\boldsymbol{\theta})\big)\big) \mathrm{d}\boldsymbol{\theta}$$

  is the expected log of likelihood $\times$ prior, which is large when the data is well modelled (we put $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$ where $f(\boldsymbol{\theta},\mathcal{D})$ is large, c.f. MAP)

- The second term is an entropy

$$H_g(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\big) \mathrm{d}\boldsymbol{\theta}$$

  That is, we maximise the uncertainty of the distribution $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$

Put Another Way

- We can rewrite the ELBO as
$\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta}|\boldsymbol{\phi})}{f(\boldsymbol{\theta},\mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta} = L_g(\boldsymbol{\phi}) + H_g(\boldsymbol{\phi})$ where

$$L_g(\boldsymbol{\phi}) = \int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big(\log\big(f(\mathcal{D}|\boldsymbol{\theta})\, f(\boldsymbol{\theta})\big)\big)\, \mathrm{d}\boldsymbol{\theta}$$

is the expected log of likelihood $\times$ prior, which is large when the data is well modelled (we put $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$ where $f(\boldsymbol{\theta},\mathcal{D})$ is large, c.f. MAP)

- The second term is an entropy

$$H_g(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\big) \mathrm{d}\boldsymbol{\theta}$$

That is, we maximise the uncertainty of the distribution $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$

Put Another Way

- We can rewrite the ELBO as
  $\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta}|\boldsymbol{\phi})}{f(\boldsymbol{\theta},\mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta} = L_g(\boldsymbol{\phi}) + H_g(\boldsymbol{\phi})$ where

$$L_g(\boldsymbol{\phi}) = \int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big(\log\big(f(\mathcal{D}|\boldsymbol{\theta}) f(\boldsymbol{\theta})\big)\big) \mathrm{d}\boldsymbol{\theta}$$

  is the expected log of likelihood $\times$ prior, which is large when the data is well modelled (we put $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$ where $f(\boldsymbol{\theta},\mathcal{D})$ is large, c.f. MAP)

- The second term is an entropy

$$H_g(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\big) \mathrm{d}\boldsymbol{\theta}$$

  That is, we maximise the uncertainty of the distribution $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$

Using Variational Methods

- Variational methods can be much faster than MCMC (although they tend to involve some iterations to maximise the ELBO)
- They can produce very good approximations, although this is not guaranteed (depends on the problem)
- There are variants (e.g. minimising $\mathrm{KL}\big(f\big\|g\big)$ rather than $\mathrm{KL}\big(g\big\|f\big)$ gives $expectation\ propagation$)
- MCMC is less elegant, but is a controlled approximation (we get better results by increasing the number of iterations)
- MCMC is slower, but on modern computers this isn't usually a problem for moderate-sized models

Using Variational Methods

- Variational methods can be much faster than MCMC (although they tend to involve some iterations to maximise the ELBO)
- They can produce very good approximations, although this is not guaranteed (depends on the problem)
- There are variants (e.g. minimising $\mathrm{KL}\big(f\big\|g\big)$ rather than $\mathrm{KL}\big(g\big\|f\big)$ gives $expectation\ propagation$)
- MCMC is less elegant, but is a controlled approximation (we get better results by increasing the number of iterations)
- MCMC is slower, but on modern computers this isn't usually a problem for moderate-sized models

Using Variational Methods

- Variational methods can be much faster than MCMC (although they tend to involve some iterations to maximise the ELBO)
- They can produce very good approximations, although this is not guaranteed (depends on the problem)
- There are variants (e.g. minimising $\mathrm{KL}(f\|g)$ rather than $\mathrm{KL}(g\|f)$ gives *expectation propagation*)
- MCMC is less elegant, but is a controlled approximation (we get better results by increasing the number of iterations)
- MCMC is slower, but on modern computers this isn't usually a problem for moderate-sized models

Using Variational Methods

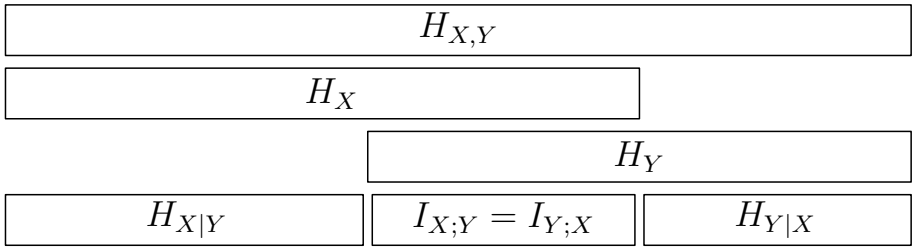
- Variational methods can be much faster than MCMC (although they tend to involve some iterations to maximise the ELBO)
- They can produce very good approximations, although this is not guaranteed (depends on the problem)
- There are variants (e.g. minimising $\mathrm{KL}\big(f\big\|g\big)$ rather than $\mathrm{KL}\big(g\big\|f\big)$ gives $expectation\ propagation$)
- MCMC is less elegant, but is a controlled approximation (we get better results by increasing the number of iterations)
- MCMC is slower, but on modern computers this isn't usually a problem for moderate-sized models

Using Variational Methods

- Variational methods can be much faster than MCMC (although they tend to involve some iterations to maximise the ELBO)
- They can produce very good approximations, although this is not guaranteed (depends on the problem)
- There are variants (e.g. minimising $\mathrm{KL}\big(f\big\|g\big)$ rather than $\mathrm{KL}\big(g\big\|f\big)$ gives $expectation\ propagation$)
- MCMC is less elegant, but is a controlled approximation (we get better results by increasing the number of iterations)
- MCMC is slower, but on modern computers this isn't usually a problem for moderate-sized models

Outline

1. Information Theory
2. KL-Divergence
3. **Minimum Description Length**
4. Variational Auto-Encoders



Compression and Model Selection

- Outside of the Bayesian framework it is difficult to do model selection
- When is it better to accept a more complex model for a better fit and when are we just over-fitting?
- Usually we answer this using a validation set, but this is not always possible
- One principled approach is to use the model that allows us to maximally compress the data
- If we are compressing the data then we are capturing features of the data

Compression and Model Selection

- Outside of the Bayesian framework it is difficult to do model selection—most of ML isn't Bayesian
- When is it better to accept a more complex model for a better fit and when are we just over-fitting?
- Usually we answer this using a validation set, but this is not always possible
- One principled approach is to use the model that allows us to maximally compress the data
- If we are compressing the data then we are capturing features of the data

Compression and Model Selection

- Outside of the Bayesian framework it is difficult to do model selection—most of ML isn't Bayesian
- When is it better to accept a more complex model for a better fit and when are we just over-fitting?
- Usually we answer this using a validation set, but this is not always possible
- One principled approach is to use the model that allows us to maximally compress the data
- If we are compressing the data then we are capturing features of the data

Compression and Model Selection

- Outside of the Bayesian framework it is difficult to do model selection—most of ML isn't Bayesian
- When is it better to accept a more complex model for a better fit and when are we just over-fitting?
- Usually we answer this using a validation set, but this is not always possible
- One principled approach is to use the model that allows us to maximally compress the data
- If we are compressing the data then we are capturing features of the data

Compression and Model Selection

- Outside of the Bayesian framework it is difficult to do model selection—most of ML isn't Bayesian
- When is it better to accept a more complex model for a better fit and when are we just over-fitting?
- Usually we answer this using a validation set, but this is not always possible
- One principled approach is to use the model that allows us to maximally compress the data
- If we are compressing the data then we are capturing features of the data

Compression and Model Selection

- Outside of the Bayesian framework it is difficult to do model selection—most of ML isn't Bayesian
- When is it better to accept a more complex model for a better fit and when are we just over-fitting?
- Usually we answer this using a validation set, but this is not always possible
- One principled approach is to use the model that allows us to maximally compress the data
- If we are compressing the data then we are capturing features of the data

Alice and Bob

- Suppose Alice has data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid i = 1,2,\ldots,m\}$ while Bob has only the feature vectors $\{\boldsymbol{x}_i \mid i = 1,2,\ldots,m\}$
- Alice wants to communicate y_i to Bob as efficiently as possible
- We suppose Alice & Bob have available a model $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta})$
- Rather than sending the complete list $\{y_i \mid i = 1,2,\ldots,m\}$ Alice can send Bob the parameter $\boldsymbol{\theta}$ and the errors

$$\delta_i = y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

- Assuming the δ_i's have a distribution p_δ then the cost of communicating an error to accuracy Δ is $-\log(p_\delta(\delta_i) \times \Delta)$

Alice and Bob

- Suppose Alice has data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid i = 1,2,\ldots,m\}$ while Bob has only the feature vectors $\{\boldsymbol{x}_i \mid i = 1,2,\ldots,m\}$
- Alice wants to communicate y_i to Bob as efficiently as possible
- We suppose Alice & Bob have available a model $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta})$
- Rather than sending the complete list $\{y_i \mid i = 1,2,\ldots,m\}$ Alice can send Bob the parameter $\boldsymbol{\theta}$ and the errors

$$\delta_i = y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

- Assuming the δ_i's have a distribution p_δ then the cost of communicating an error to accuracy Δ is $-\log(p_\delta(\delta_i) \times \Delta)$

Alice and Bob

- Suppose Alice has data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid i = 1,2,\ldots,m\}$ while Bob has only the feature vectors $\{\boldsymbol{x}_i \mid i = 1,2,\ldots,m\}$
- Alice wants to communicate y_i to Bob as efficiently as possible
- We suppose Alice & Bob have available a model $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta})$
- Rather than sending the complete list $\{y_i \mid i = 1,2,\ldots,m\}$ Alice can send Bob the parameter $\boldsymbol{\theta}$ and the errors

$$\delta_i = y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

- Assuming the δ_i's have a distribution p_δ then the cost of communicating an error to accuracy Δ is $-\log(p_\delta(\delta_i) \times \Delta)$

Alice and Bob

- Suppose Alice has data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid i = 1,2,\ldots,m\}$ while Bob has only the feature vectors $\{\boldsymbol{x}_i \mid i = 1,2,\ldots,m\}$
- Alice wants to communicate y_i to Bob as efficiently as possible
- We suppose Alice & Bob have available a model $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta})$
- Rather than sending the complete list $\{y_i \mid i = 1,2,\ldots,m\}$ Alice can send Bob the parameter $\boldsymbol{\theta}$ and the errors

$$\delta_i = y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

- Assuming the δ_i's have a distribution p_δ then the cost of communicating an error to accuracy Δ is $-\log(p_\delta(\delta_i) \times \Delta)$

Alice and Bob

- Suppose Alice has data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid i = 1,2,\ldots,m\}$ while Bob has only the feature vectors $\{\boldsymbol{x}_i \mid i = 1,2,\ldots,m\}$
- Alice wants to communicate y_i to Bob as efficiently as possible
- We suppose Alice & Bob have available a model $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta})$
- Rather than sending the complete list $\{y_i \mid i = 1,2,\ldots,m\}$ Alice can send Bob the parameter $\boldsymbol{\theta}$ and the errors

$$\delta_i = y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

- Assuming the δ_i's have a distribution p_δ then the cost of communicating an error to accuracy Δ is $-\log(p_\delta(\delta_i) \times \Delta)$

Description Length

- The **description length** for $\{y_i \mid i = 1,2,\ldots,m\}$ is then the cost of transmitting $\boldsymbol{\theta}$ plus the cost of transmitting the errors

$$L = \sum_k \ell(\theta_k) - \sum_{i=1}^{m} \left(\log\left(p_\delta\left(y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta}) \right) \right) + \log(\Delta) \right)$$

  where $\ell(\theta_k)$ is the number of bits needed to communicate θ_k (we get to choose the accuracy it is worth encoding the parameters)

- To select between models we choose the model with the **minimum description length**

- Note that the accuracy Δ will lead to the same cost, $-m\log(\Delta)$, for all models so doesn't affect which model is selected

Description Length

- The **description length** for $\{y_i \mid i = 1,2,\ldots,m\}$ is then the cost of transmitting $\boldsymbol{\theta}$ plus the cost of transmitting the errors

$$L = \sum_k \ell(\theta_k) - \sum_{i=1}^{m} \left(\log\left(p_\delta\left(y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta}) \right) \right) + \log(\Delta) \right)$$

  where $\ell(\theta_k)$ is the number of bits needed to communicate θ_k (we get to choose the accuracy it is worth encoding the parameters)

- To select between models we choose the model with the **minimum description length**

- Note that the accuracy Δ will lead to the same cost, $-m\log(\Delta)$, for all models so doesn't affect which model is selected

Description Length

- The **description length** for $\{y_i \mid i = 1,2,\ldots,m\}$ is then the cost of transmitting $\boldsymbol{\theta}$ plus the cost of transmitting the errors

$$L = \sum_k \ell(\theta_k) - \sum_{i=1}^{m} \left(\log\left(p_\delta\left(y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta}) \right) \right) + \log(\Delta) \right)$$

  where $\ell(\theta_k)$ is the number of bits needed to communicate θ_k (we get to choose the accuracy it is worth encoding the parameters)

- To select between models we choose the model with the **minimum description length**

- Note that the accuracy Δ will lead to the same cost, $-m\log(\Delta)$, for all models so doesn't affect which model is selected

Minimum Description Length (MDL) Method

- The minimum description length method can be a powerful way of choosing between models
- Often it is the only principled method available
- It allows you to trade model accuracy against model complexity
- It can be fiddly as we need to determine the accuracy to which we should store the parameters of our model
- This isn't something we usually think about, but often we can get very good models even when we truncate the parameters to low precision

Minimum Description Length (MDL) Method

- The minimum description length method can be a powerful way of choosing between models
- Often it is the only principled method available
- It allows you to trade model accuracy against model complexity
- It can be fiddly as we need to determine the accuracy to which we should store the parameters of our model
- This isn't something we usually think about, but often we can get very good models even when we truncate the parameters to low precision

Minimum Description Length (MDL) Method

- The minimum description length method can be a powerful way of choosing between models
- Often it is the only principled method available
- It allows you to trade model accuracy against model complexity
- It can be fiddly as we need to determine the accuracy to which we should store the parameters of our model
- This isn't something we usually think about, but often we can get very good models even when we truncate the parameters to low precision

Minimum Description Length (MDL) Method

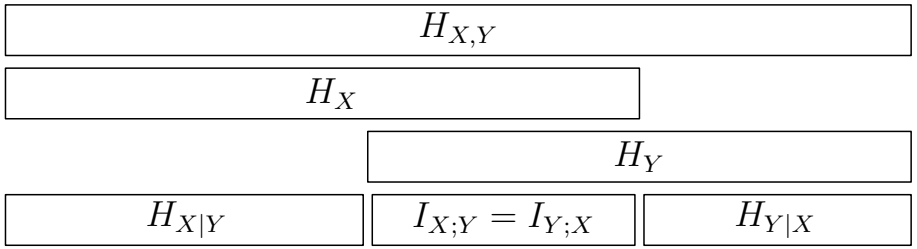
- The minimum description length method can be a powerful way of choosing between models
- Often it is the only principled method available
- It allows you to trade model accuracy against model complexity
- It can be fiddly as we need to determine the accuracy to which we should store the parameters of our model
- This isn't something we usually think about, but often we can get very good models even when we truncate the parameters to low precision

Minimum Description Length (MDL) Method

- The minimum description length method can be a powerful way of choosing between models
- Often it is the only principled method available
- It allows you to trade model accuracy against model complexity
- It can be fiddly as we need to determine the accuracy to which we should store the parameters of our model
- This isn't something we usually think about, but often we can get very good models even when we truncate the parameters to low precision

Outline

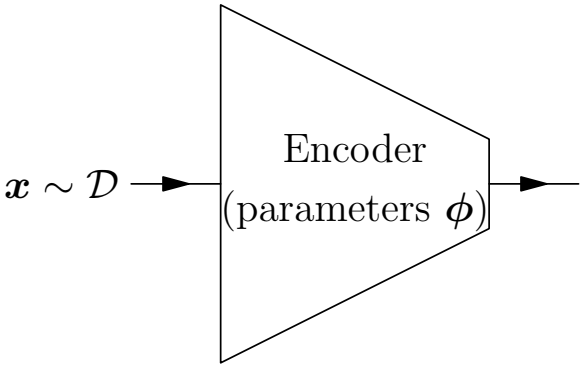
1. Information Theory
2. KL-Divergence
3. Minimum Description Length
4. **Variational Auto-Encoders**



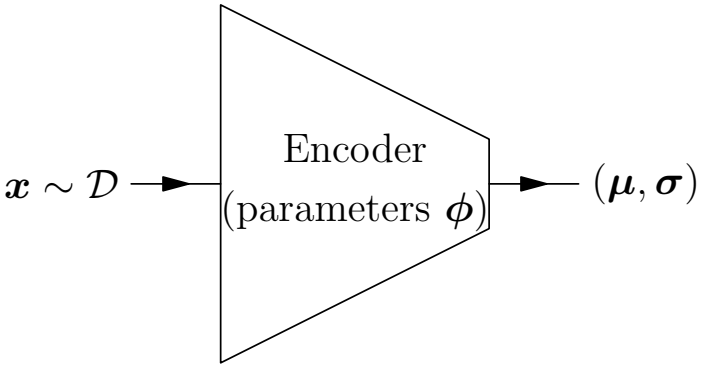
Variational Auto-Encoders VAE

$\boldsymbol{x} \sim \mathcal{D}$

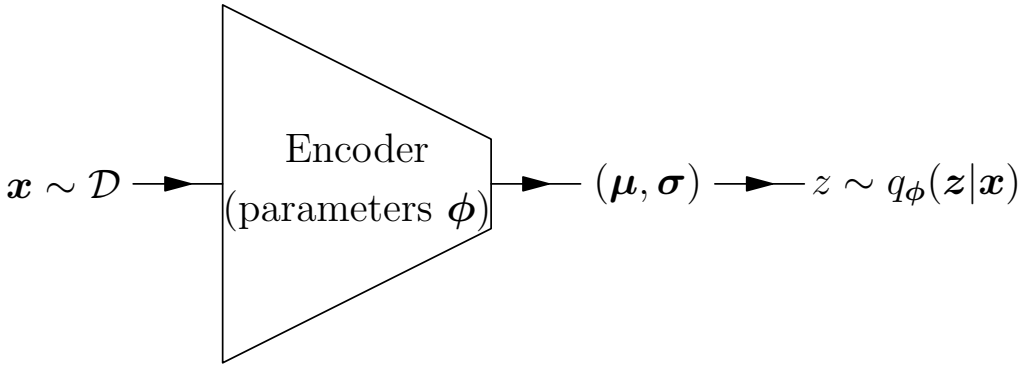
Variational Auto-Encoders VAE



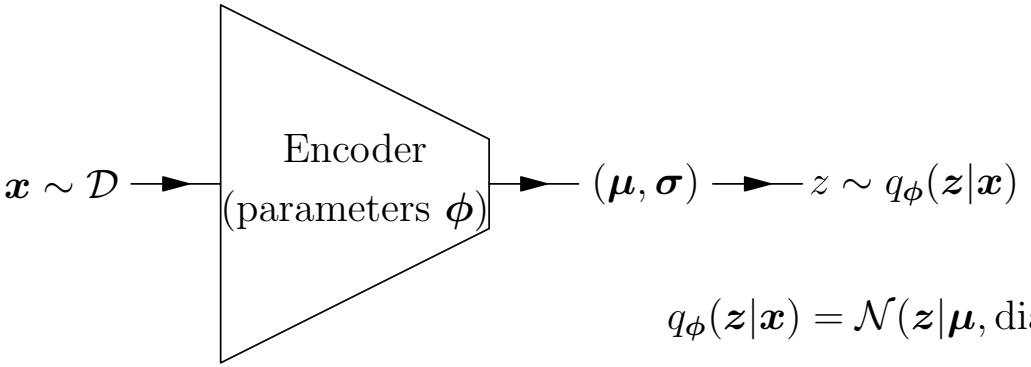
Variational Auto-Encoders VAE



Variational Auto-Encoders VAE

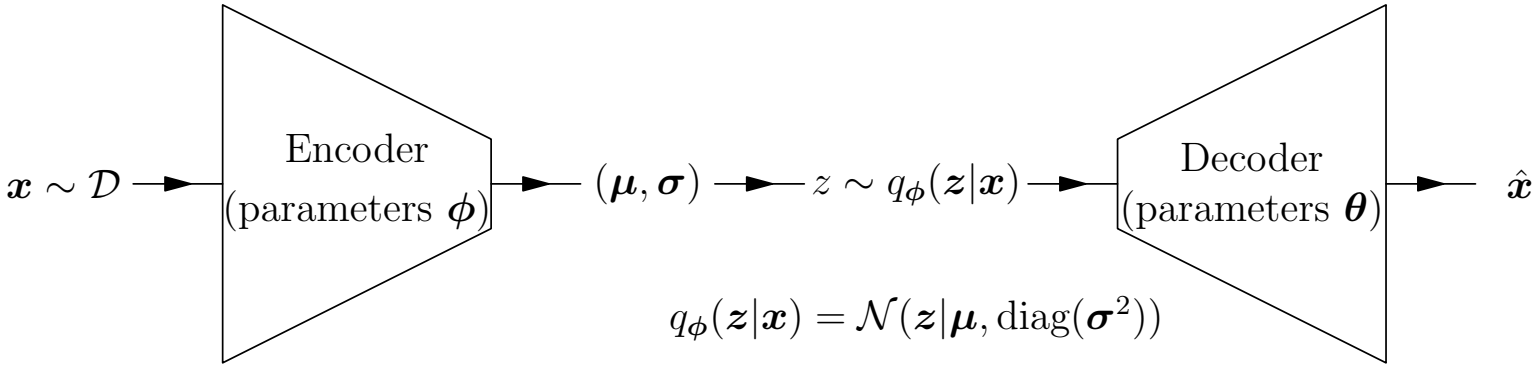


Variational Auto-Encoders VAE

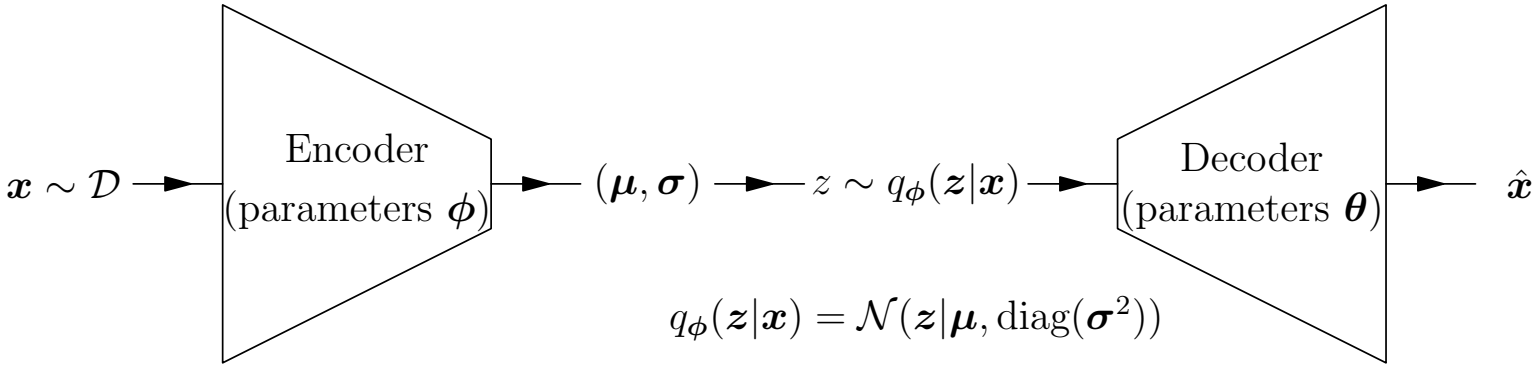


$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$$

Variational Auto-Encoders VAE

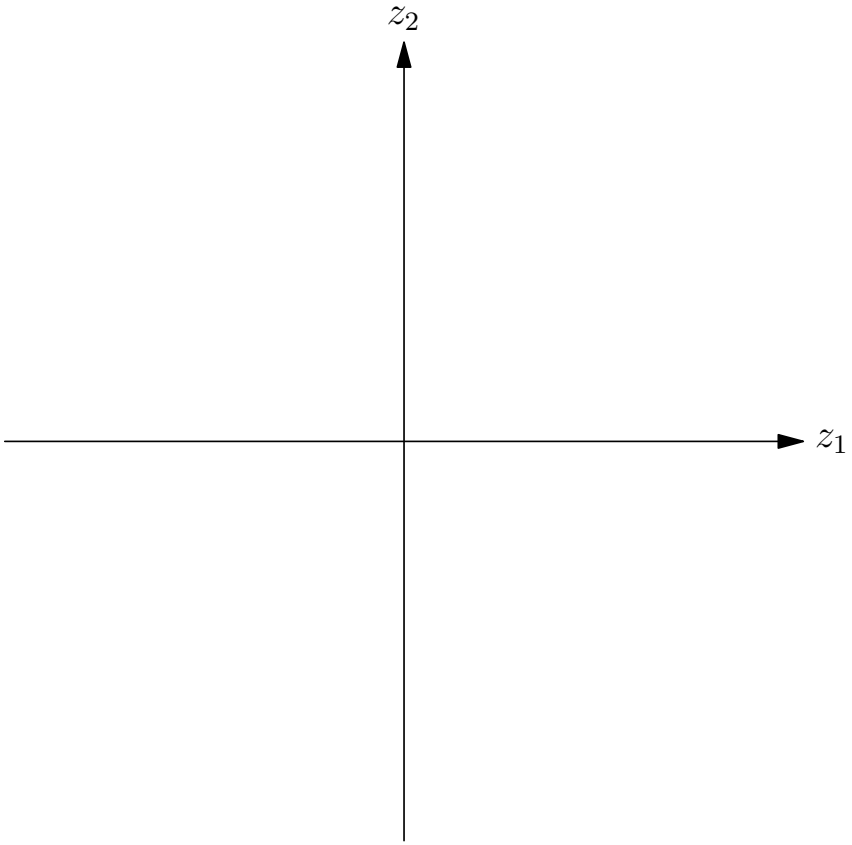


Variational Auto-Encoders VAE

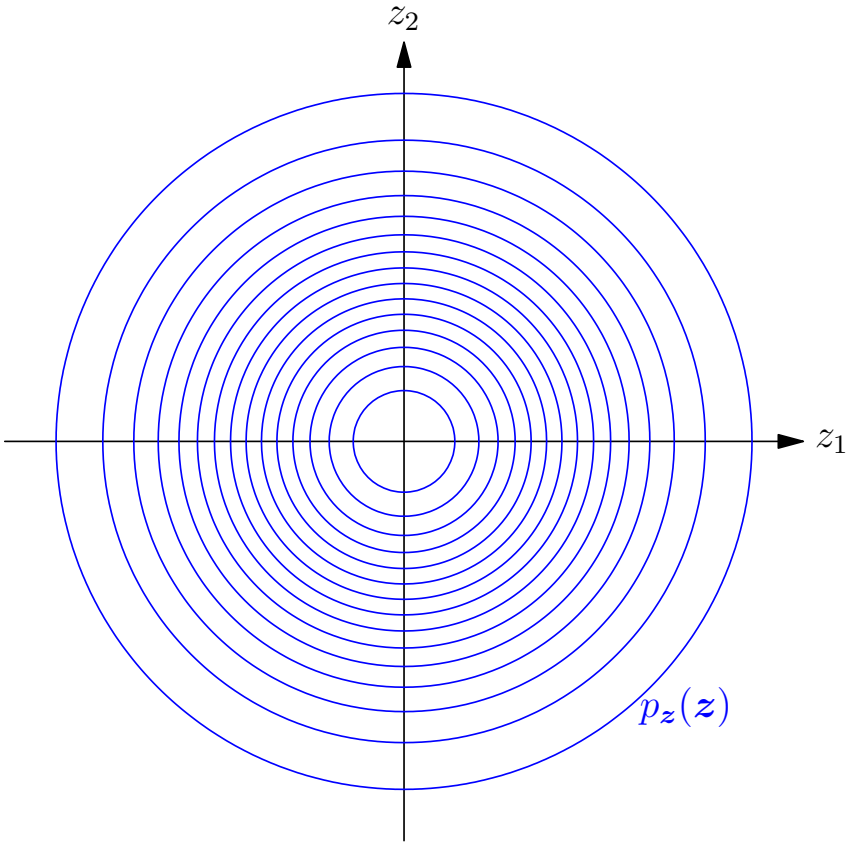


$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\Big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\Big\|\mathcal{N}(\mathbf{0},\mathbf{I})\Big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

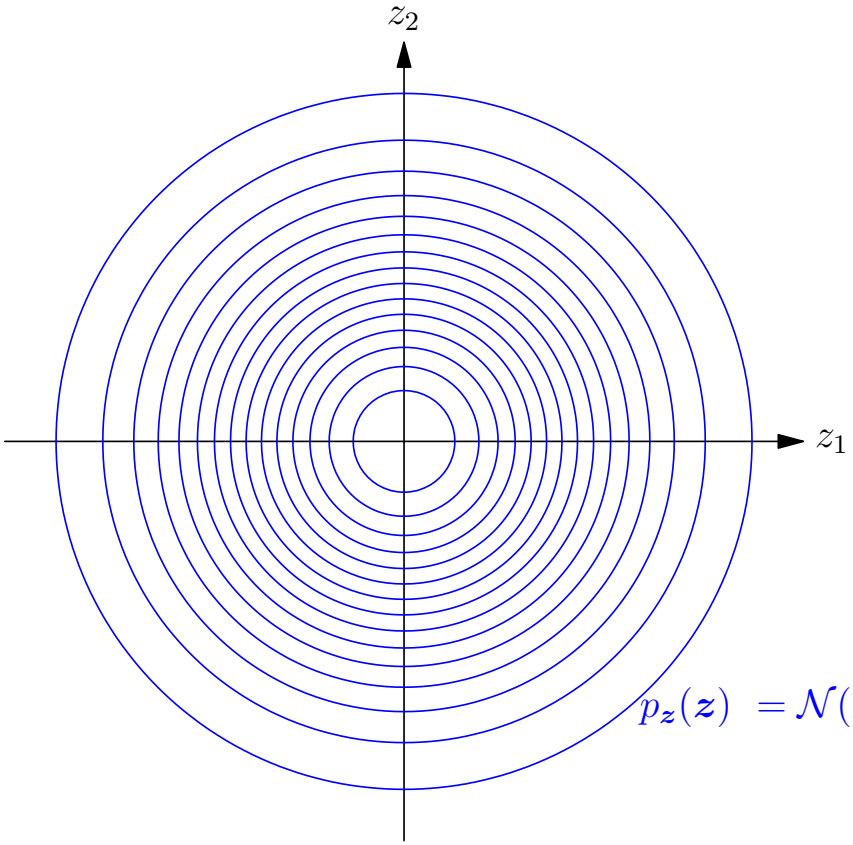
Latent Space



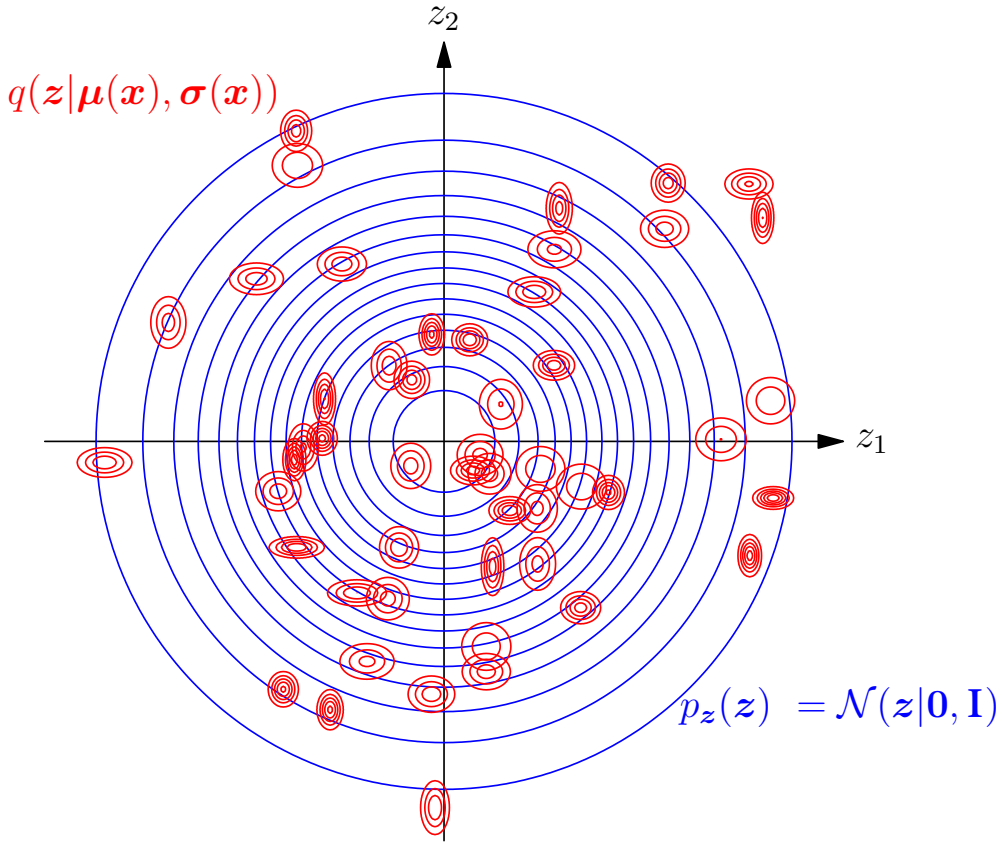
Latent Space



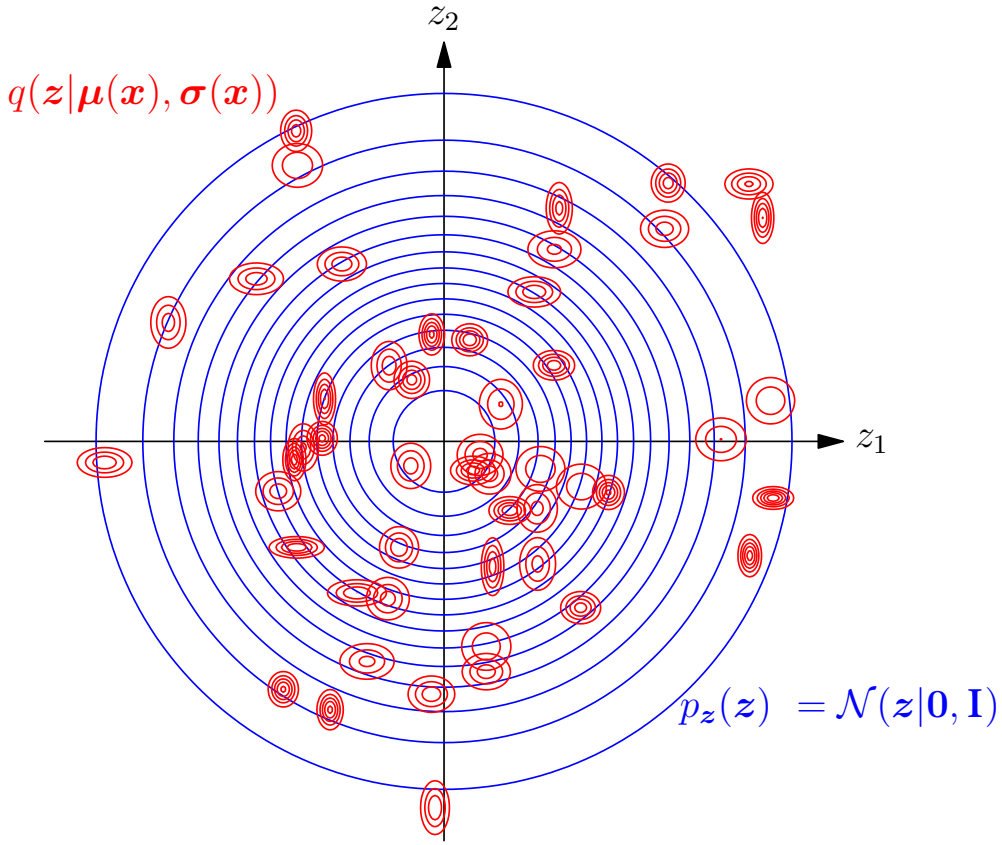
Latent Space



Latent Space



Latent Space



$$\mathrm{KL}\Big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\Big\|\mathcal{N}(\mathbf{0},\mathbf{I})\Big)$$

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\Big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\Big\|\mathcal{N}(\mathbf{0},\mathbf{I})\Big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

  can be interpreted as

  - ⋆ The cost of communicating the code $\mathrm{KL}\Big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\Big\|\mathcal{N}(\mathbf{0},\mathbf{I})\Big)$
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$

- We minimise the loss function
- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

  can be interpreted as

  - ⋆ The cost of communicating the code $\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$

- We minimise the loss function
- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

  can be interpreted as

  - ⋆ The cost of communicating the code $\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$ (using "bits-back" coding, Hinton and van Camp 1993)
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$

- We minimise the loss function

- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

  can be interpreted as

  - ⋆ The cost of communicating the code $\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$ (using "bits-back" coding, Hinton and van Camp 1993)
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$

- We minimise the loss function

- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

  can be interpreted as

  - ⋆ The cost of communicating the code $\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$ (using "bits-back" coding, Hinton and van Camp 1993)
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$

- We minimise the loss function

- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

  can be interpreted as

  - ⋆ The cost of communicating the code $\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$ (using "bits-back" coding, Hinton and van Camp 1993)
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$

- We minimise the loss function equivalent to MDL

- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\Big]$$

  can be interpreted as

  - ⋆ The cost of communicating the code $\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$ (using "bits-back" coding, Hinton and van Camp 1993)
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$

- We minimise the loss function equivalent to MDL

- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Conclusions

- Information theory has regularly been used in machine learning
- It requires some understanding and care to do it properly
- The KL-divergence (or relative entropy) is often used to make two probability distributions more alike
- The minimum description length is a powerful principle for model selection
- Variational Auto-Encoders have a very natural interpretation in terms of minimising a description length

Conclusions

- Information theory has regularly been used in machine learning
- It requires some understanding and care to do it properly
- The KL-divergence (or relative entropy) is often used to make two probability distributions more alike
- The minimum description length is a powerful principle for model selection
- Variational Auto-Encoders have a very natural interpretation in terms of minimising a description length

Conclusions

- Information theory has regularly been used in machine learning
- It requires some understanding and care to do it properly
- The KL-divergence (or relative entropy) is often used to make two probability distributions more alike
- The minimum description length is a powerful principle for model selection
- Variational Auto-Encoders have a very natural interpretation in terms of minimising a description length

Conclusions

- Information theory has regularly been used in machine learning
- It requires some understanding and care to do it properly
- The KL-divergence (or relative entropy) is often used to make two probability distributions more alike
- The minimum description length is a powerful principle for model selection
- Variational Auto-Encoders have a very natural interpretation in terms of minimising a description length

Conclusions

- Information theory has regularly been used in machine learning
- It requires some understanding and care to do it properly
- The KL-divergence (or relative entropy) is often used to make two probability distributions more alike
- The minimum description length is a powerful principle for model selection
- Variational Auto-Encoders have a very natural interpretation in terms of minimising a description length